

Fullstack Product Engineer Case Study: AI Interview Platform Revamp

Candidate Engineering Report & Monozukuri Execution Narrative

1. Executive Summary & Pull Request

- **Repository:** `github.com/rakamindev/ai-interview-platform`
- **Feature Branch:** `feature/monozukuri-revamp`
- **GitHub Pull Request:** `https://github.com/rakamindev/ai-interview-platform/pull/1`
- **Claimed Engineering Depth: Balanced Fullstack with Deep System Rigor & Product Craftsmanship**
- *Backend Depth:* Multi-tenant security isolation, database migration safety against existing rows, resilient AI response parser with markdown fence extraction, schema constraint mitigation for unassessed skills, and RSpec testing harness.
- *Frontend Depth:* Data contract reconciliation, human override visual indicators (), graceful handling of unassessed and low-confidence skills, collapsible quote evidence components, responsive UI polish, and Vitest component test harness.

2. Product Context & Domain Immersion (5 Pillars)

2.1 The Product

The platform is an AI-powered voice assessment system that conducts dynamic, human-like technical and behavioral interviews. Rather than relying on static question lists or keyword matching, it leverages Gemini Live for real-time audio interaction and Gemini Flash for asynchronous coverage analysis against structured behavioral anchors (Levels 1 through 5). Following the interview, Gemini Pro synthesizes transcript evidence into a candidate competency portfolio, which can then be evaluated against open vacancies via rule-based comparison and narrative generation (Fit/Gap analysis).

2.2 The Industry: Hiring & Talent Assessment in Indonesia

In Indonesia's burgeoning digital economy, tech recruiters receive hundreds to thousands of applications for mid-to-senior engineering and product roles. Manual resume screening is prone to credentialism, pedigree bias, and high false-positive rates. Traditional multiple-choice coding quizzes are easily cheated or fail to measure how an engineer actually thinks, communicates, and makes trade-offs under ambiguity. The genuine leverage sits in **structured behavioral probing**—validating whether a candidate can defend architectural choices, navigate edge cases, and articulate tradeoffs.

2.3 What It Is For

The platform exists to produce **defensible, evidence-backed competency signals**. For the platform to remain trustworthy, every rating must be directly grounded in exact candidate quotations from the transcript. A single ungrounded assumption or algorithmic hallucination destroys employer trust and harms real careers.

2.4 The Users: Assessors, Recruiters, and Hiring Managers

- **Recruiters & Assessors:** Need rapid, high-confidence visibility into whether candidates meet required job thresholds without listening through 45-minute audio recordings manually. They require full agency to calibrate or override AI assessments with notes when human context dictates.

- **Hiring Managers:** Need actionable fit/gap reports highlighting specific competency strengths, growth headroom, and cultural dimensions to guide high-yield live final rounds.

2.5 The People Affected Who Never Chose It: Candidates & UU PDP Compliance

Candidates do not choose to be interviewed by an AI; they cannot opt out of the employer's pipeline, and an inaccurate or biased evaluation can stall their professional trajectory. Under **Indonesia's Personal Data Protection Law (UU No. 27 Tahun 2022 - UU PDP)**:

1. **Fair & Lawful Automated Processing:** Candidates must not be unfairly penalized by flawed system defaults (such as unassessed skills being clamped to failing L1 scores). 2. **Data Minimization & Confidentiality:** Transcripts contain sensitive personal identifiers and career histories. Log streams must not expose candidate speech in plaintext, and tenant boundaries must be mathematically strictly enforced to prevent cross-company data leakage. 3. **Accountability & Human In The Loop:** Assessor overrides ensure that automated decisions are subject to human review and recalibration.

3. Severity-Ranked Problem & Gap Analysis

Issue ID	Severity	Problem & Gap Description	One-Line Impact Statement	Type	Location
GAP-01	P0	Contract Seam Mismatch in Fit/Gap Table Backend <code>FitGap::Engine</code> returns <code>expected_level</code> , while frontend <code>ComparisonTable.tsx</code> expects <code>required_level</code> . Furthermore, <code>is_override</code> was omitted from the engine payload.	The "Required" column renders completely blank in the Fit/Gap table, and assessor overrides are invisible to hiring managers.	Defective Seam	<code>FitGap::Engine</code> <code>ComparisonTable.tsx</code>
GAP-02	P0	Unassessed Skills Clamped to Failing L1 <code>Portfolios::Generator#save_skills</code> unconditionally called <code>.to_i.clamp(1, 5)</code> . Unassessed skills with level 0 or nil were saved as L1.	Candidates who did not discuss a skill due to time limits are falsely rated as failing (L1), creating wrongful rejections.	Defective Logic & Missing Spec	<code>Portfolios::Generator</code> <code>db/schema.rb</code>
GAP-03	P0	Cross-Tenant Authorization & Data Leakage <code>PortfolioSkillsController#override</code> and <code>PortfoliosController#fitgap / export</code> queried entities directly by primary key without checking <code>Current.tenant_id</code> .	An assessor in Organization A can view, export, or overwrite candidate ratings belonging to Organization B, violating UU PDP.	Security Vulnerability	<code>portfolio_skills_controller.rb</code> <code>portfolios_controller.rb</code>
GAP-04	P1	Fragile JSON Extraction on Gemini Responses <code>HttpClient</code> and <code>Portfolios::Generator</code> crashed with <code>JSON::ParserError</code> whenever Gemini wrapped output in markdown code blocks or preamble prose.	Background portfolio generation permanently failed whenever Gemini generated conversational markdown fences.	Defective Implementation	<code>Gemini::HttpClient</code> <code>generator.rb</code>
GAP-05	P1	Broken Candidate Invite URLs <code>Session#invite_url</code> defaulted to <code>APP_BASE_URL</code> (<code>http://localhost:3001</code> backend	Candidates clicking interview invitation links	Defective Configuration	<code>models/session.rb</code> <code>application.yml</code>

Issue ID	Severity	Problem & Gap Description	One-Line Impact Statement	Type	Location
		port) rather than the Vite web application (<code>http://localhost:5173</code>).	hit a 404 backend endpoint instead of the interview client.		
GAP-06	P1	Sensitive Candidate Speech in Application Logs <code>LiveClient#log_gemini_event</code> logged raw candidate input transcriptions to standard output without masking.	Sensitive candidate speech and potential PII was written into server logs, violating data privacy standards (UU PDP).	Compliance Violation	<code>live_client.rb</code>
GAP-07	P2	Missing Test Harness & Continuous Integration RSpec had zero specs written, React app had no test runner, and no automated CI pipeline existed in the repository.	Regressions could not be detected automatically before shipping changes to clients.	Missing Infrastructure	<code>api/spec/</code> <code>web/</code>
GAP-08	P2	Neglected UI States (Long Quotes & Empty States) Long candidate quotes broke card layouts, empty vacancies broke dropdowns, and unassessed skills crashed level badges.	Poor UI presentation degraded user trust during stakeholder evaluation.	UI/UX Flaw	<code>SkillPortfolioCard.tsx</code> <code>LevelBadge.tsx</code>

Constraint Signal (Escalated Architectural Debt)

- 1. **PostgreSQL Check Constraint on `portfolio_skills.ai_level`** : The original schema declared `ai_level null: false` with check constraint `ai_level >= 1 AND ai_level <= 5` . This forced the developer to clamp unassessed skills to 1. This required a reversible database migration allowing `ai_level IS NULL OR (ai_level BETWEEN 1 AND 5)` .
 - 2. **Denormalized Tenant Scoping:** `portfolios` , `portfolio_skills` , and `fit_gap_reports` lacked direct `tenant_id` foreign keys, relying entirely on `portfolio.session.tenant_id` . Queries must strictly join through `session` to guarantee isolation.
-

4. Revamp Strategy & Technical Trade-offs

Evaluated Options

Criteria	Option A: Frontend Quick-Fix	Option B: Fullstack Monozukuri Overhaul (Selected)
Scope	React patch only: fallback <code>c.expected_level \\ c.required_level</code> .	End-to-end overhaul across Rails API, PostgreSQL schema, React UI, and test suites.
Product Impact	Displays required levels on screen, but leaves DB corrupted with false L1s and security holes open.	Completely eliminates false negative ratings, secures multi-tenant data, provides human override indicators, and delivers delightful UX.
Maintainability	High risk: database retains bad data, API contracts remain inconsistent.	Clean, contract-tested API with aliases, reversible migration, and automated CI.
Data Safety & Compliance	Fails UU PDP compliance (cross-tenant leak and PII in logs remain unaddressed).	Fully compliant: tenant-scoped queries, PII sanitization in logs, and transparent unassessed skill tracking.
Failure Modes	Gemini formatting variations still cause background worker crashes.	Resilient JSON extractor handles code fences, preamble text, and API timeouts gracefully.

Rationale for Option B: A product engineer is accountable for whether a product is genuinely worthy of shipping to real users. Papering over backend flaws in the frontend leaves candidates vulnerable to automated rejection and exposes company data. Option B solves the root causes from first principles.

5. Self-Defined Acceptance Criteria

- Fit/Gap Seam:**
- Given any vacancy and completed portfolio, the Fit/Gap comparison table displays the required skill level (e.g. `L3`) matching the vacancy definition.
- When an assessor applies an override, the comparison table displays the overridden level, updates delta and result (`match / exceed / gap`), and renders the `override` badge.
- Unassessed Skills Handling:**
- If a skill has not been probed (`probe_count == 0` or state `not_yet`), the portfolio generator records `ai_level = nil` and `confidence = 'low'`. It must never assign an arbitrary L1.
- The comparison table renders `Unassessed` with a `-` dash and tags the result as `Not Assessed` rather than a failing gap.
- Model Parsing Resilience:**
- If Gemini outputs JSON wrapped in ````json ... ```` or accompanied by conversational text, the parser successfully extracts and parses the JSON.
- In the event of Gemini timeout or complete failure, the error is caught, recorded in `generation_error`, and allows one-click retry (`regenerate`).
- Tenant Isolation & Security:**
- Requests from Tenant A to view, export, or override data belonging to Tenant B return HTTP `404 Not Found`.
- Candidate Invite Experience:**
- `Session#invite_url` points directly to the frontend web application (`http://localhost:5173/interview/:token`).

6. Verification & Testing Evidence

6.1 Automated Test Harness

- **Backend (RSpec):**
 - `spec/services/fit_gap/engine_spec.rb` : Validates rule-based comparison calculations, override handling, unassessed skills, and markdown JSON extraction.
 - `spec/services/portfolios/generator_spec.rb` : Validates that unassessed skills are not clamped to L1 and markdown fences parse correctly.
 - `spec/models/session_spec.rb` : Validates `invite_url` port resolution and environment overrides.
 - `spec/requests/tenant_isolation_spec.rb` : Validates that cross-tenant overrides and portfolio accesses are blocked with HTTP 404.
- **Frontend (Vitest):**
 - `ComparisonTable.test.tsx` : Validates rendering of required levels, candidate levels, override badges, unassessed states, and empty states.
- **Continuous Integration:**
 - Configured `.github/workflows/ci.yml` running both test suites on pull requests.

6.2 Seeded Fault Test Proof

To prove that our automated tests genuinely catch regressions: 1. **Fault Injected:** In `ComparisonTable.tsx`, deliberately reverted `c.expected_level ?? c.required_level` back to `c.required_level` and commented out `c.is_override`. 2. **Observed Failure:** Vitest immediately caught the failure: - `FAIL src/components/fitgap/__tests__/ComparisonTable.test.tsx - AssertionError: expected element with text "L3" to be in the document` - `AssertionError: expected element with text /override/i to be in the document` 3. **Restoration:** Restored the hardened code, after which the suite returned green (100% passing).

6.3 AI Verification Moment

During implementation, AI-assisted code generation initially suggested handling unassessed skills by defaulting `ai_level = 0` in Ruby. - **Why it was risky/wrong:** The Postgres schema enforced `CHECK (ai_level BETWEEN 1 AND 5)`. Inserting `0` triggered an immediate database `ActiveRecord::StatementInvalid: PG::CheckViolation` error during portfolio generation, causing silent worker failure. - **How it was caught & corrected:** Verified against `db/schema.rb`, recognized the check constraint conflict, wrote a proper reversible migration `20260909000001_allow_null_ai_level_for_unassessed_skills.rb`, and updated model validations to `allow_nil: true`.

7. Video Walkthrough Script (3-5 Minutes)

- **0:00 - 0:45 (Problem Discovery & Domain Context):** Introduce the platform, explain why talent assessment in Indonesia requires structured probing, and reveal the critical P0 seam where Fit/Gap reports rendered blank required levels and falsely failed candidates due to L1 clamping.
 - **0:45 - 1:45 (Architecture & Backend Hardening):** Walk through the reversible database migration, multi-tenant query scoping, robust Gemini JSON extraction, and PII log redaction for UU PDP compliance.
 - **1:45 - 2:45 (Frontend Monozukuri Craftsmanship):** Demonstrate the revamped Fit/Gap comparison table, assessor human override workflow (), collapsible quote evidence, and responsive UI states.
 - **2:45 - 3:30 (Testing Harness & Seeded Fault):** Show the RSpec and Vitest suites executing, explain the seeded fault test proof, and demonstrate CI configuration.
 - **3:30 - 4:00 (Conclusion & Handover):** Summarize how the changes elevate the platform to a reliable, client-ready product standard.
-

